



UNIVERSIDAD DE LAS FUERZAS ARMADAS
ESPE

CONSTRUCCIÓN Y EVOLUCIÓN DEL SOFTWARE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Metodologías de Desarrollo de Software

Estudiante:

Ednan Josué Merino Calderón

Docente:

Ing. Ángel Marcelo Rea Guamán Ph.D

Índice general

1. Antecedentes	2
2. Objetivo	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Desarrollo	4
3.1. Waterfall (Cascada)	4
3.1.1. Detalle, área de mejor desempeño y tipo de proyectos	4
3.1.2. Ventajas	4
3.1.3. Desventajas	4
3.2. Scrum	5
3.2.1. Detalle, área de mejor desempeño y tipo de proyectos	5
3.2.2. Ventajas	5
3.2.3. Desventajas	5
3.3. Kanban	6
3.3.1. Detalle, área de mejor desempeño y tipo de proyectos	6
3.3.2. Ventajas	6
3.3.3. Desventajas	6
3.4. Lean Software Development	7
3.4.1. Detalle, área de mejor desempeño y tipo de proyectos	7
3.4.2. Ventajas	7
3.4.3. Desventajas	7
4. Conclusiones	8

1. Antecedentes

El desarrollo de software ha experimentado una transformación significativa a lo largo de las últimas décadas. En sus inicios, los proyectos se gestionaban mediante enfoques estructurados y lineales, donde la planificación detallada y la documentación exhaustiva eran los pilares fundamentales. El modelo en cascada (Waterfall) fue durante muchos años la referencia principal, al establecer una secuencia rígida de fases que debían completarse de forma ordenada, desde el análisis de requisitos hasta el mantenimiento del sistema.

Con el paso del tiempo, las necesidades del mercado y la naturaleza cambiante de los proyectos tecnológicos evidenciaron las limitaciones de estos enfoques tradicionales. La aparición de los métodos ágiles marcó un punto de inflexión al priorizar la adaptabilidad, la comunicación constante entre los equipos y la entrega continua de valor al cliente. Entre las metodologías más reconocidas se encuentran Scrum, Kanban y Lean Software Development, cada una con sus propios principios, prácticas y contextos de aplicación.

Actualmente, la industria del software combina enfoques tradicionales y ágiles para responder de manera más efectiva a los desafíos de desarrollo moderno. Este trabajo presenta un análisis de cuatro de las metodologías más utilizadas en la práctica profesional —Waterfall, Scrum, Kanban y Lean Software Development— con el propósito de comprender sus características, identificar sus ventajas y desventajas, y establecer los contextos en los que resultan más convenientes para el éxito de los proyectos.

2. Objetivo

2.1. Objetivo General

Analizar y comparar las cuatro metodologías de desarrollo de software más utilizadas en la industria actual, para identificar sus características, ventajas, desventajas y los tipos de proyectos más adecuados para su aplicación.

2.2. Objetivos Específicos

- Describir detalladamente cada metodología, su ámbito de aplicación y los tipos de proyecto en los que se utiliza.
- Evaluar las ventajas y desventajas de cada una de estas metodologías en el contexto del desarrollo de software.
- Realizar una comparación que permita facilitar en el futuro la selección de la metodología más adecuada según el tipo de proyecto, el entorno organizacional y su naturaleza.

3. Desarrollo

3.1. Waterfall (Cascada)

3.1.1. Detalle, área de mejor desempeño y tipo de proyectos

La metodología en cascada consiste en seguir un proceso secuencial de etapas: requisitos → diseño → implementación → pruebas → mantenimiento. Cada fase debe finalizar antes de pasar a la siguiente y, por lo general, no se permite regresar a fases anteriores [1, 2].

Este enfoque se desempeña mejor en entornos donde los requisitos están claramente definidos desde el inicio, no se espera que cambien o donde los cambios implican un coste elevado. Por ello, es común en proyectos gubernamentales, de infraestructura, sistemas críticos o entornos regulados (por ejemplo, banca, defensa o salud) [8].

3.1.2. Ventajas

- Claridad en la planificación y en las fases del proyecto: cada etapa está definida y los entregables también [1].
- Buen nivel de documentación, lo cual facilita rigurosidad, trazabilidad y mantenimiento [2].
- Facilita la gestión de proyectos con requisitos estables y bien comprendidos.

3.1.3. Desventajas

- Muy poca flexibilidad: si los requisitos cambian, puede generar mucho retrabajo o incluso inviabilidad del proyecto.

- Retraso en la entrega de valor: el usuario puede ver la solución solo al final del proceso, por lo tanto, el feedback llega tarde.
- Riesgo de errores tardíos: los defectos suelen detectarse en fases finales, cuando ya existe una alta inversión [1].

3.2. Scrum

3.2.1. Detalle, área de mejor desempeño y tipo de proyectos

Scrum es un marco ágil de trabajo que organiza el desarrollo en iteraciones llamadas *sprints*, generalmente de una a cuatro semanas. Se basa en los pilares de transparencia, inspección y adaptación, promoviendo equipos autoorganizados que colaboran estrechamente para alcanzar objetivos comunes [5, 6].

Es especialmente útil en proyectos donde los requisitos cambian con frecuencia, como el desarrollo web, móvil o productos innovadores. Su estructura iterativa permite entregar valor de manera incremental y adaptarse rápidamente al feedback del cliente [8].

3.2.2. Ventajas

- Alta adaptabilidad a los cambios de requerimientos gracias a los ciclos cortos de entrega [5].
- Entrega de valor frecuente, permitiendo obtener retroalimentación temprana y ajustar el rumbo [6].
- Mejora la comunicación del equipo y fomenta la participación activa de los interesados.

3.2.3. Desventajas

- Requiere un equipo comprometido y autoorganizado; sin ello, puede fracasar.
- Puede ser difícil estimar presupuesto o tiempo cuando los requisitos cambian constantemente.

- En proyectos grandes, si no se gestiona correctamente, puede perderse coordinación entre equipos [2].

3.3. Kanban

3.3.1. Detalle, área de mejor desempeño y tipo de proyectos

Kanban es una metodología visual de gestión del trabajo que se centra en optimizar el flujo de tareas y reducir los cuellos de botella. Su origen proviene del sistema de producción de Toyota y fue adaptado posteriormente al desarrollo de software. [3].

Se aplica con éxito en proyectos de mantenimiento, soporte y mejora continua, donde las tareas llegan de forma constante y no hay iteraciones predefinidas como en Scrum. También se adapta bien a equipos que desean optimizar su proceso actual sin hacer cambios radicales [7].

3.3.2. Ventajas

- Visualización clara del trabajo: el equipo puede observar el estado de cada tarea mediante tableros [3].
- Permite adaptarse al cambio e incorporar nuevas tareas de forma flexible sin necesidad de reiniciar ciclos completos.
- Control del flujo de trabajo al limitar el trabajo en progreso (WIP), lo que reduce los tiempos de entrega [7].

3.3.3. Desventajas

- Menos estructura temporal: al no existir sprints fijos, puede perderse la disciplina o cadencia si el equipo no tiene experiencia [3].
- Dificultad para estimar entregas a largo plazo en entornos con alta variabilidad.
- Requiere seguimiento constante de métricas (tiempo de ciclo, flujo, productividad) para lograr mejoras reales [7].

3.4. Lean Software Development

3.4.1. Detalle, área de mejor desempeño y tipo de proyectos

El enfoque de Lean Software Development (LSD) deriva de los principios del pensamiento Lean aplicados originalmente a la manufactura. Su propósito principal es eliminar desperdicios, optimizar el flujo de valor, potenciar el aprendizaje y empoderar al equipo [4].

Se utiliza principalmente en entornos donde la eficiencia y la rapidez son esenciales, como startups, proyectos de innovación o equipos que buscan mejorar procesos existentes. Promueve la entrega temprana y continua de valor, evitando tareas que no contribuyen directamente al producto final [9].

3.4.2. Ventajas

- Reducción del desperdicio: se eliminan actividades que no generan valor, lo que mejora la eficiencia general [4].
- Entrega más rápida de valor al cliente mediante la optimización del flujo continuo.
- Fortalece la autonomía del equipo y promueve una cultura de mejora continua [9].

3.4.3. Desventajas

- Requiere una cultura organizacional madura, con equipos comprometidos con la mejora constante [4].
- La flexibilidad excesiva puede generar ambigüedad en los objetivos o entregables si no se gestionan adecuadamente.
- Puede provocar sobrecarga de optimización si no se equilibra con las metas reales del proyecto.

4. Conclusiones

El análisis realizado permitió obtener una visión clara sobre la evolución y aplicación de las metodologías de desarrollo de software más representativas en la actualidad. A través del estudio de Waterfall, Scrum, Kanban y Lean Software Development se comprendió cómo cada una responde a distintos contextos de gestión, tipos de proyecto y estilos de trabajo en equipo dentro del ámbito tecnológico.

Las metodologías tradicionales, como Waterfall, mantienen su vigencia en proyectos con requerimientos bien definidos y estructuras formales que demandan control y documentación precisa. En contraste, los enfoques ágiles como Scrum, Kanban y Lean se adaptan mejor a entornos dinámicos donde los requisitos cambian con frecuencia y la colaboración constante es esencial para alcanzar los objetivos.

Del análisis comparativo se desprende que cada metodología ofrece ventajas y limitaciones propias. Mientras Waterfall garantiza orden y trazabilidad, Scrum fomenta la innovación continua y la retroalimentación rápida. Kanban optimiza el flujo de trabajo y la eficiencia operativa, y Lean impulsa una cultura de mejora constante centrada en la reducción de desperdicios y la entrega de valor real al cliente.

En conjunto, se evidencia que la selección de una metodología depende del contexto del proyecto, el grado de incertidumbre, la madurez del equipo y los objetivos estratégicos de la organización. La tendencia actual orienta hacia la adopción de modelos híbridos, que integran la estructura de los métodos tradicionales con la flexibilidad y adaptabilidad de los ágiles. Esta combinación permite equilibrar la planificación con la capacidad de respuesta, p

Bibliografía

- [1] Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería del software: Un enfoque práctico* (8a ed.). McGraw-Hill Education.
- [2] Sommerville, I. (2016). *Ingeniería del Software* (10a ed.). Pearson Educación.
- [3] Anderson, D. J. (2010). *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press.
- [4] Poppendieck, M., & Poppendieck, T. (2003). *Lean Software Development: An Agile Toolkit*. Addison-Wesley.
- [5] Scrum Alliance. (s.f.). *¿Qué es Scrum?*. Recuperado de <https://www.scrumalliance.org/about-scrum>
- [6] Amazon Web Services. (s.f.). *¿Qué es Scrum?*. Recuperado de <https://aws.amazon.com/what-is-scrum/>
- [7] Radigan, D. (s.f.). *Kanban: How the Kanban methodology applies to software development*. Atlassian. Recuperado de <https://www.atlassian.com/agile/kanban>
- [8] Lucidchart Blog. (s.f.). *Agile vs. Waterfall vs. Kanban vs. Scrum*. Recuperado de <https://www.lucidchart.com/blog/agile-vs-waterfall-vs-kanban-vs-scrum>
- [9] Lutkevich, B., & Silverthorne, V. (2021, diciembre 6). *Lean software development*. TechTarget. Recuperado de <https://www.techtarget.com/searchsoftwarequality/definition/lean-programming>